Introduction

Tasks

- Write a function that converts bound constraints to general stage constraints.
- Write a function that converts stage constraints to trajectory constraints.
- Write a function that computes the constraints for a QP arising in tracking problems.
- Implement a receding horizon controller with constraints and non-zero target state.

Framework provided:

myStageConstraints.m	Template for submission
myTrajectoryConstraints.m	Template for submission
myConstraintMatrices.m	Template for submission
myMPController.m	Template for submission
SimscapeCrane_MPChard.slx	Nonlinear Simscape model for testing controller
SSmodelParams.mat	Physical parameters for the linear ODE
Params_Simscape.mat	Physical parameters for nonlinear simulation
GantryResponsePlot.m	Utility function
MatlabSimulation.m	Simulates the linear gantry crane model
testMyMPC.m	Function to test your RHC law performance
genCraneODE.p	Answer function for a previous assignment
genPrediction.p	Answer function for a previous assignment
genCostMatrices.p	Answer function for a previous assignment

Approach

Suppose you have a horizon length N and a model of a discrete-time system of the form

$$x_{k+1} = Ax_k + Bu_k,\ k = 0, 1, \ldots, N-1 \tag{1}$$

where the state $x_k \in \mathbb{R}^n$ and input $u_k \in \mathbb{R}^m$ at sample instant k. Suppose also that a target state x_e is given and that x_e is an equilibrium point if the input applied to the system is zero, i.e. $x_e = Ax_e$.

- Edit **myStageConstraints.m** to compute matrices $\bar{D}$ (`Dt`), $\bar{E}$ (`Et`) and vector $\bar{b}$ (`bt`) such that

  $$\bar{D}x_k + \bar{E}u_k \leq \bar{b},\ k = 0, 1 \ldots, N-1 \tag{2}$$

  is equivalent to

  $$\check{u} \leq u_k \leq \hat{u},\ k = 0, 1 \ldots, N-1 \tag{3a}$$
  $$\check{c} \leq Dx_k \leq \hat{c},\ k = 1, 2, \ldots, N \tag{3b}$$

  where $\check{u}$ (`ul`) and $\check{c}$ (`cl`) are lower bounds, $\hat{u}$(`uh`) and $\hat{c}$ (`ch`) are upper bounds and the constrained vector $Dx_k \in \mathbb{R}^c$ is defined by the matrix D.

  Your code should ensure that the number of constraints in (2) is equal to the number of constraints in (3).

- Edit **myTrajectoryConstraints.m** to compute matrices **D** (`DD`), **E** (`EE`) and **b** (`bb`) such that (2) is equivalent to

$$\mathbf{D}\tilde{\mathbf{x}} + \mathbf{E}\mathbf{u} \leq \mathbf{b} \tag{4}$$

where $\tilde{\mathbf{x}} := [x_0^T \; x_1^T \; \cdots \; x_{N-1}^T]^T$ and $\mathbf{u} := [u_0^T \; u_1^T \; \cdots \; u_{N-1}^T]^T$. Note that $\mathbf{b}$ is *not* a function of x_0.

Your code should ensure that the number of constraints in (2) is equal to the number of constraints in (4).

- Edit **myConstraintMatrices.m** to compute the matrices F, J and L in the quadratic program (QP) given by

$$\mathbf{u}^*(x_0, x_e) := \arg\min_{\mathbf{u}} \; \|x_N - x_e\|_P^2 + \sum_{k=0}^{N-1} \left(\|x_k - x_e\|_Q^2 + \|u_k\|_R^2\right) \text{ s.t. (1)–(2)} \tag{5a}$$

$$= \arg\min_{\mathbf{u}} \frac{1}{2}\mathbf{u}^T H \mathbf{u} + (x_0 - x_e)^T G^T \mathbf{u} \text{ s.t. } F\mathbf{u} \leq \mathbf{b} + Jx_0 + Lx_e \tag{5b}$$

where P, Q and R are given positive semi-definite matrices and $\mathbf{x} = \Phi x_0 + \Gamma \mathbf{u}$ as in the previous assignment. The notation $\|z\|_M^2 := z^T M z$ for any square matrix M.

- We will use the MATLAB function `mpcqpsolver` to solve the QPs arising in this course. Read the documentation for `mpcqpsolver` and `mpcqpsolverOptions`. Please pay special attention to input/output conventions and the way inequality constraints are posed in `mpcqpsolver`, since these are different to (5b). See **testMyMPC.m** for details on putting H into an appropriate form for `mpcqpsolver` as discussed in the MATLAB documentation.

- Edit **myMPController.m** so that it provides the value of the RHC law κ for the target state x_e and the current estimate of the state $\hat{x} =: x_0$, i.e.

$$\kappa(\hat{x}, x_e) := u_0^*(\hat{x}, x_e) \tag{6a}$$

where

$$\mathbf{u}^*(\hat{x}, x_e) =: \left[u_0^*(\hat{x}, x_e)^T \; u_1^*(\hat{x}, x_e)^T \; \cdots \; u_{N-1}^*(\hat{x}, x_e)^T\right]^T. \tag{6b}$$

Please see how the function is called from the test script **testMyMPC.m**. Pay special attention to which matrices are inside and which are outside the function.

- Once all of the above functions are in working order you will be able to use **testMyMPC.m** for assessing the performance of your controller. Open **SimscapeCrane_MPChard.slx**, navigate to the block *SimscapeCrane_MPChard/MPC* and replace the code inside the MATLAB function block with your own. Investigate the effect of using different cost weights, sample period, horizon length, initial conditions, target state, constraints, etc. This will be helpful for the final assignment.

Submission

filename	Description.
myStageConstraints.m	Computes the matrices and vector in (2)
myTrajectoryConstraints.m	Computes the matrices and vector in (4)
myConstraintMatrices.m	Computes the constraint matrices for the QP (5b)
myMPController.m	Computes the optimal RHC input (6a) with a non-zero target equilibrium state by solving the QP (5b)

Edit or upload the MATLAB template for the four files above into Cody Coursework. All your code should be written for the general case and not just for the gantry crane.

- Cody Coursework
- Course
- People
- Catalog
- Eric Kerrigan
  - Log Out
- Help

- Course Details expand all
- Mathematical Model of a Gantry Crane
  - State space representation
  - Problem Explore
- Receding Horizon Control
  - Prediction Matrices
  - Cost Function Matrices
  - Linear RHC Law Without Constraints
  - Problem Explore
- RHC with Hard Constraints
  - Stage Constraints
  - Trajectory Constraints
  - QP Constraint Matrices
  - Receding Horizon Controller
  - Problem Explore
- RHC with Soft Constraints
  - Augmenting QP Matrices and Vectors
  - Problem Explore
- Assignment

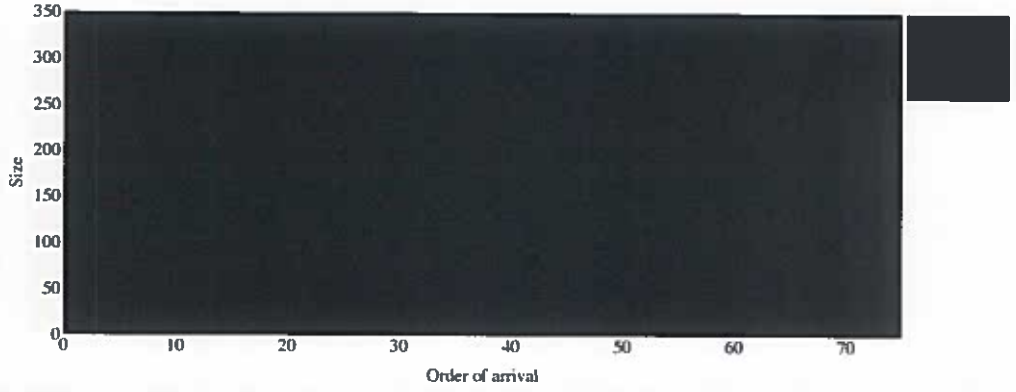
Stage Constraints

Student Solutions

Search by student name, solution ID, or function Search by student name, solution ID, or function

- List
- Map

View: All Solutions Test Solutions Submitted Solutions



Previous

2nd Submission: All tests passed

Test Results

Submitted on 3 Feb 2017 by Jun Yu | Solution ID: 3422107

```
function [Dt,Et,bt]=myStageConstraints(A,B,D,cl,ch,ul,uh)
[n,m]=size(B);
Dt=[D*A;-D*A;zeros(2*m,n)];
Et=[D*B;-D*B;eye(m);-eye(m)];
bt=[ch;-cl;uh;-ul];
end
```

Next

View solutions submitted via the Submit button.

- Cody Coursework
- Course
- People
- Catalog
- Eric Kerrigan
  - Log Out
- Help

- Course Details expand all
- Mathematical Model of a Gantry Crane
  - State space representation
  - Problem Explore
- Receding Horizon Control
  - Prediction Matrices
  - Cost Function Matrices
  - Linear RHC Law Without Constraints
  - Problem Explore
- RHC with Hard Constraints
  - Stage Constraints
  - Trajectory Constraints
  - QP Constraint Matrices
  - Receding Horizon Controller
  - Problem Explore
- RHC with Soft Constraints
  - Augmenting QP Matrices and Vectors
  - Problem Explore
- Assignment

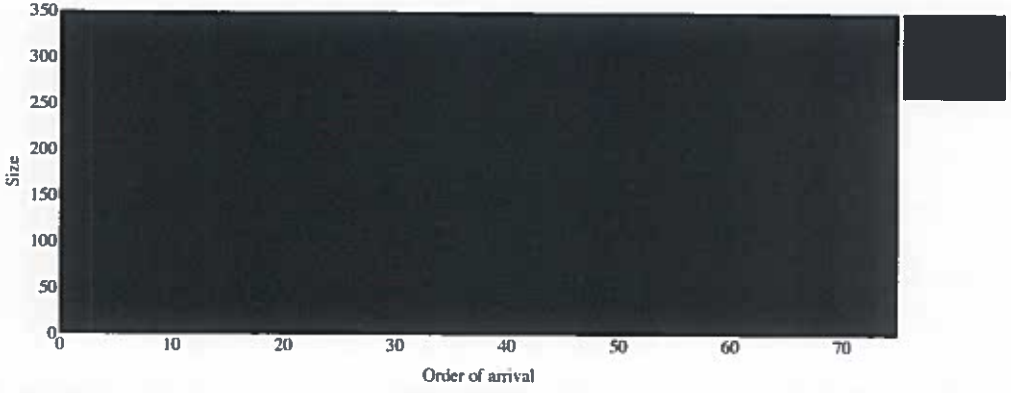
Stage Constraints

Student Solutions

Search by student name, solution ID, or function Search by student name, solution ID, or function

- List
- Map

View: All Solutions Test Solutions Submitted Solutions



Previous

3rd Submission: All tests passed

Test Results

Submitted on 4 Feb 2017 by ANASTASIS GEORGIOU | Solution ID: 3428449

```
function [Dt,Et,bt]=myStageConstraints(A,B,D,cl,ch,ul,uh)

n = size(A,2);
m = size(B,2);
c = size(D,1);

O = zeros(m,n);
I = eye(m);

Dt = zeros([2*(c+m)),n);
Et = zeros([2*(c+m)),m);
bt = zeros([2*(c+m)),1);

Dt([1:c],:)= D*A;
Dt([c+1:2*c],:)= -(D*A);
Dt([2*c+1:2*c+m],:)= O;
Dt([2*c+m+1:2*(c+m)],:)= O;

Et([1:c],:)= D*B;
Et([c+1:2*c],:)= -(D*B);
Et([2*c+1:2*c+m],:)= I;
Et([2*c+m+1:2*(c+m)],:)= -I;

bt([1:c],1)= ch;
bt([c+1:2*c],1)= -cl;
bt([2*c+1:2*c+m],1)= uh;
bt([2*c+m+1:2*(c+m)],1)= -ul;

end
```

Next

- Course Details expand all
- Mathematical Model of a Gantry Crane
  - State space representation
  - Problem Explore
- Receding Horizon Control
  - Prediction Matrices
  - Cost Function Matrices
  - Linear RHC Law Without Constraints
  - Problem Explore
- RHC with Hard Constraints
  - Stage Constraints
  - Trajectory Constraints
  - QP Constraint Matrices
  - Receding Horizon Controller
  - Problem Explore
- RHC with Soft Constraints
  - Augmenting QP Matrices and Vectors
  - Problem Explore
- Assignment

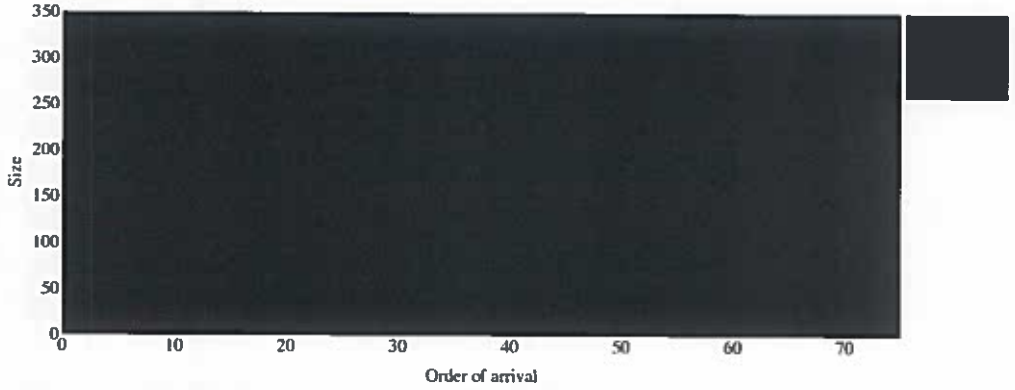
Stage Constraints

Student Solutions

Search by student name, solution ID, or function Search by student name, solution ID, or function

- List
- Map

View: All Solutions Test Solutions Submitted Solutions



Previous

3rd Submission: All tests passed

Test Results

Submitted on 14 Feb 2017 by YIQI LU | Solution ID: 3560839

```
function [Dt,Et,bt]=myStageConstraints(A,B,D,cl,ch,ul,uh)

Dt=[D*A;-D*A;zeros(size(uh,1),size(A,2));zeros(size(ul,1),size(A,2))];
Et=[D*B;-D*B;eye(size(uh,1),size(B,2));-eye(size(ul,1),size(B,2))];
bt=[ch;-cl;uh;-ul];

end
```

Next

- Cody Coursework
- Course
- People
- Catalog
- Eric Kerrigan
  - Log Out
- Help
- Course Details expand all
- Mathematical Model of a Gantry Crane
  - State space representation
  - Problem Explore
- Receding Horizon Control
  - Prediction Matrices
  - Cost Function Matrices
  - Linear RHC Law Without Constraints
  - Problem Explore
- RHC with Hard Constraints
  - Stage Constraints
  - Trajectory Constraints
  - QP Constraint Matrices
  - Receding Horizon Controller
  - Problem Explore
- RHC with Soft Constraints
  - Augmenting QP Matrices and Vectors
  - Problem Explore
- Assignment

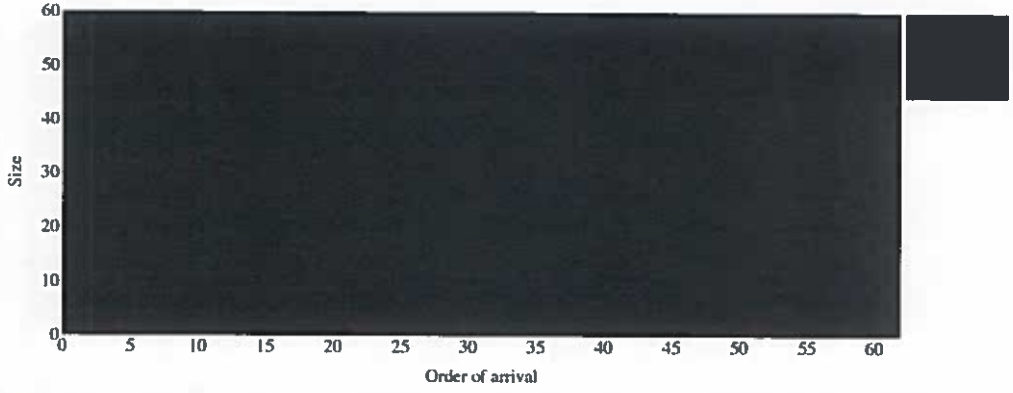
Trajectory Constraints

Student Solutions

Search by student name, solution ID, or function Search by student name, solution ID, or function

- List
- Map

View: ◉ All Solutions ○ Test Solutions ○ Submitted Solutions



Previous

1st Submission: All tests passed

Test Results

Submitted on 10 Feb 2017 by CAILONG HUA | Solution ID: 3513670

```
function [DD,EE,bb]=myTrajectoryConstraints(Dt,Et,bt,N)

% your code goes here
IN=eye(N,N);
N1=ones(N,1);
DD=kron(IN,Dt);
EE=kron(IN,Et);
bb=kron(N1,bt);
end
```

Next

- Cody Coursework
- Course
- People
- Catalog
- Eric Kerrigan
  - Log Out
- Help
- Course Details expand all
- Mathematical Model of a Gantry Crane
  - State space representation
  - Problem Explore
- Receding Horizon Control
  - Prediction Matrices
  - Cost Function Matrices
  - Linear RHC Law Without Constraints
  - Problem Explore
- RHC with Hard Constraints
  - Stage Constraints
  - Trajectory Constraints
  - QP Constraint Matrices
  - Receding Horizon Controller
  - Problem Explore
- RHC with Soft Constraints
  - Augmenting QP Matrices and Vectors
  - Problem Explore
- Assignment

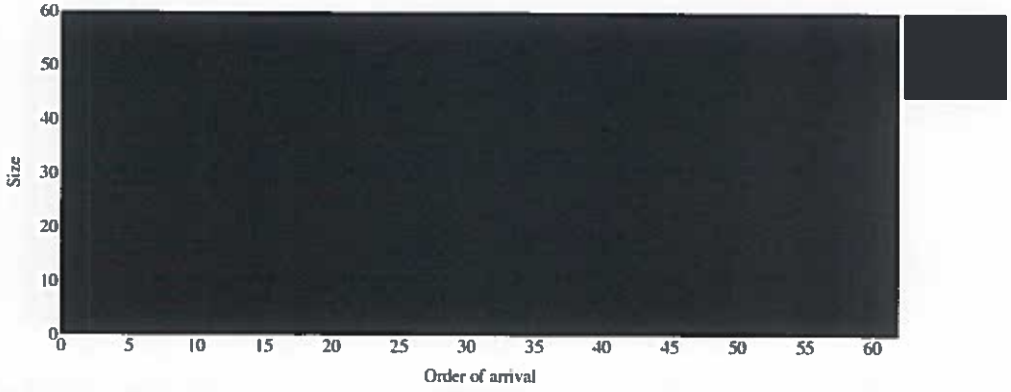
Trajectory Constraints

Student Solutions

Search by student name, solution ID, or function Search by student name, solution ID, or function

- List
- Map

View: All Solutions Test Solutions Submitted Solutions



Previous

1st Submission: All tests passed

Test Results

Submitted on 5 Feb 2017 by youwen li | Solution ID: 3439000

```
function [DD,EE,bb]=myTrajectoryConstraints(Dt,Et,bt,N)

% your code goes here
DD=kron(eye(N),Dt);
EE=kron(eye(N),Et);
bb=kron(ones(N,1),bt);
end
```

Next

- Cody Coursework
- Course
- People
- Catalog
- Eric Kerrigan
  - Log Out
- Help
- Course Details expand all
- Mathematical Model of a Gantry Crane
  - State space representation
  - Problem Explore
- Receding Horizon Control
  - Prediction Matrices
  - Cost Function Matrices
  - Linear RHC Law Without Constraints
  - Problem Explore
- RHC with Hard Constraints
  - Stage Constraints
  - Trajectory Constraints
  - QP Constraint Matrices
  - Receding Horizon Controller
  - Problem Explore
- RHC with Soft Constraints
  - Augmenting QP Matrices and Vectors
  - Problem Explore
- Assignment

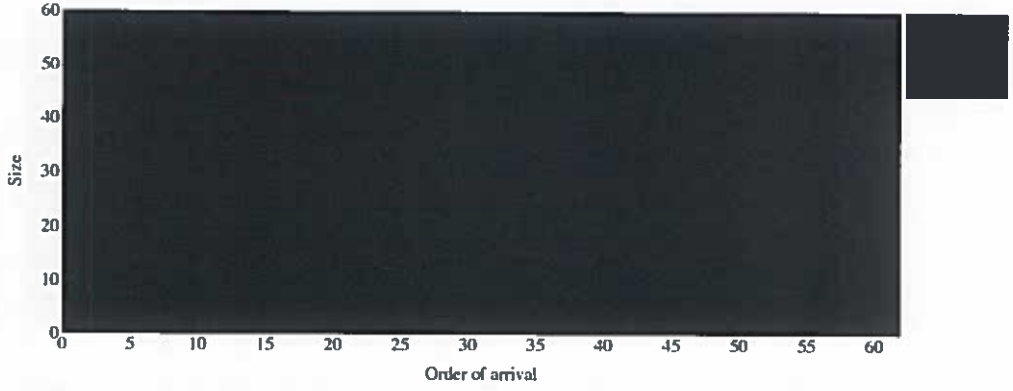
Trajectory Constraints

Student Solutions

Search by student name, solution ID, or function Search by student name, solution ID, or function

- List
- Map

View: All Solutions Test Solutions Submitted Solutions



Previous

1st Submission: All tests passed

Test Results

Submitted on 8 Feb 2017 by Alina Walch | Solution ID: 3490069

```
function [DD,EE,bb]=myTrajectoryConstraints(Dt,Et,bt,N)

% your code goes here
DD = kron(eye(N),Dt);
EE = kron(eye(N),Et);
bb = repmat(bt, N,1);

end
```

Next

- Cody Coursework
- Course
- People
- Catalog
- Eric Kerrigan
  - Log Out
- Help

- Course Details expand all
- Mathematical Model of a Gantry Crane
  - State space representation
  - Problem Explore
- Receding Horizon Control
  - Prediction Matrices
  - Cost Function Matrices
  - Linear RHC Law Without Constraints
  - Problem Explore
- RHC with Hard Constraints
  - Stage Constraints
  - Trajectory Constraints
  - QP Constraint Matrices
  - Receding Horizon Controller
  - Problem Explore
- RHC with Soft Constraints
  - Augmenting QP Matrices and Vectors
  - Problem Explore
- Assignment

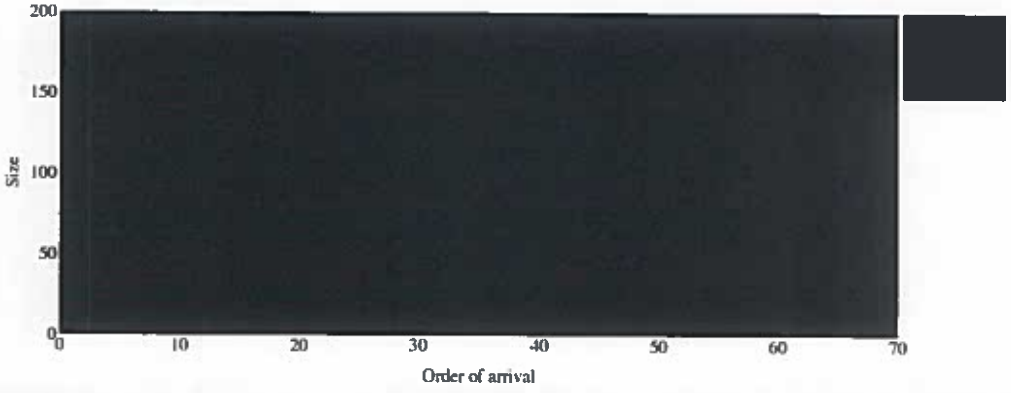
QP Constraint Matrices

Student Solutions

Search by student name, solution ID, or function Search by student name, solution ID, or function

- List
- Map

View: All Solutions Test Solutions Submitted Solutions



Previous

3rd Submission: All tests passed

Test Results

Submitted on 10 Feb 2017 by ANASTASIS GEORGIOU | Solution ID: 3514246

```
function [F,J,L]=myConstraintMatrices(DD,EE,Gamma,Phi,N)

n=size(Phi,2);
Nm=size(Gamma,2);

Phi_t = zeros(N*n,n);
Gamma_t = zeros(N*n,Nm);
I=eye(n);
O=zeros(n,Nm);

Phi_t([1:n],:)= I;
Phi_t([n+1:N*n],:)= Phi([1:N*n-n],:);

Gamma_t([1:n],:)=O;
Gamma_t([n+1:N*n],:)= Gamma([1:N*n-n],:);

F=(DD*Gamma_t+EE);
J=-(DD*Phi_t);
L= -J - DD*kron(ones(N,1),eye(n));
end
```

Next

- Cody Coursework
- Course
- People
- Catalog
- Eric Kerrigan
  - Log Out
- Help
- Course Details expand all
- Mathematical Model of a Gantry Crane
  - State space representation
  - Problem Explore
- Receding Horizon Control
  - Prediction Matrices
  - Cost Function Matrices
  - Linear RHC Law Without Constraints
  - Problem Explore
- RHC with Hard Constraints
  - Stage Constraints
  - Trajectory Constraints
  - QP Constraint Matrices
  - Receding Horizon Controller
  - Problem Explore
- RHC with Soft Constraints
  - Augmenting QP Matrices and Vectors
  - Problem Explore
- Assignment

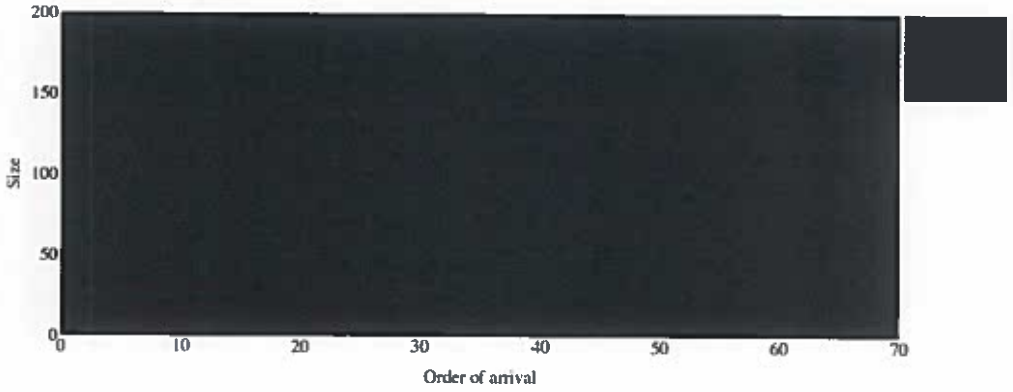
QP Constraint Matrices

Student Solutions

Search by student name, solution ID, or function Search by student name, solution ID, or function

- List
- Map

View: All Solutions Test Solutions Submitted Solutions



Previous

1st Submission: All tests passed

Test Results

Submitted on 10 Feb 2017 by Will Roe | Solution ID: 3515887

```
function [F,J,L]=myConstraintMatrices(DD,EE,Gamma,Phi,N)

% your code goes here
[Nn,Nm] = size(Gamma);

n = Nn/N;
m = Nm/N;

OneL = ones(N,1);
IL = eye(n,n);
OneL = kron(OneL,IL);

Zg = zeros(n,N*m);
GammaTilda = [Zg;Gamma(1:n*(N-1),:)];

PhiTilda = [IL;Phi(1:n*(N-1),:)];


F = DD*GammaTilda + EE;
J = -DD*PhiTilda;
L = -J - DD*(OneL);

end
```

Next

- Cody Coursework
- Course
- People
- Catalog
- Eric Kerrigan
  - Log Out
- Help
- Course Details expand all
- Mathematical Model of a Gantry Crane
  - State space representation
  - Problem Explore
- Receding Horizon Control
  - Prediction Matrices
  - Cost Function Matrices
  - Linear RHC Law Without Constraints
  - Problem Explore
- RHC with Hard Constraints
  - Stage Constraints
  - Trajectory Constraints
  - QP Constraint Matrices
  - Receding Horizon Controller
  - Problem Explore
- RHC with Soft Constraints
  - Augmenting QP Matrices and Vectors
  - Problem Explore
- Assignment

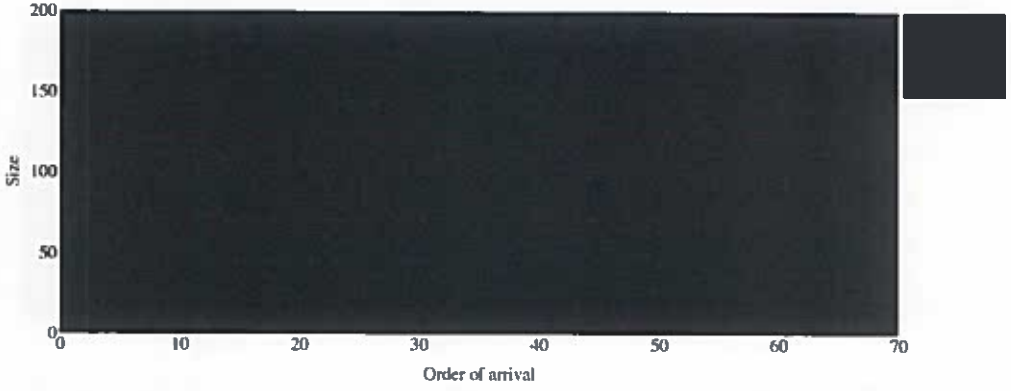
QP Constraint Matrices

Student Solutions

Search by student name, solution ID, or function Search by student name, solution ID, or function

- List
- Map

View: All Solutions Test Solutions Submitted Solutions



Previous

1st Submission: All tests passed

Test Results

Submitted on 15 Feb 2017 by Kassem El Cheikh Ali | Solution ID: 3585115

```
function [F,J,L]=myConstraintMatrices(DD,EE,Gamma,Phi,N)
n = size(Phi,2);
F = DD * [zeros(n,size(Gamma,2));Gamma(1:(N-1)*n,:)]+EE;
J = -DD*[eye(n);Phi(1:(N-1)*n,:)];
L = -J - DD*kron(ones(N,1),eye(n));
end
```

Next

- Cody Coursework
- Course
- People
- Catalog
- Eric Kerrigan
  - Log Out
- Help
- Course Details expand all
- Mathematical Model of a Gantry Crane
  - State space representation
  - Problem Explore
- Receding Horizon Control
  - Prediction Matrices
  - Cost Function Matrices
  - Linear RHC Law Without Constraints
  - Problem Explore
- RHC with Hard Constraints
  - Stage Constraints
  - Trajectory Constraints
  - QP Constraint Matrices
  - Receding Horizon Controller
  - Problem Explore
- RHC with Soft Constraints
  - Augmenting QP Matrices and Vectors
  - Problem Explore
- Assignment

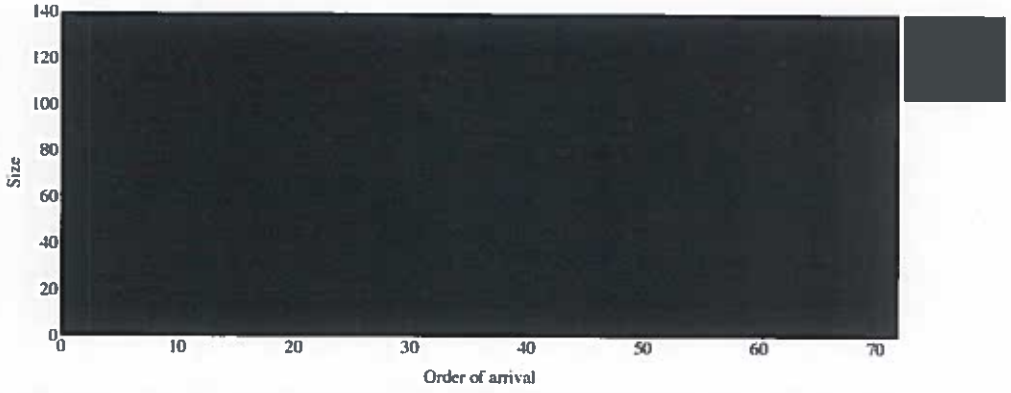
Receding Horizon Controller

Student Solutions

Search by student name, solution ID, or function Search by student name, solution ID, or function

- List
- Map

View: ◉ All Solutions ○ Test Solutions ○ Submitted Solutions



Previous

2nd Submission: All tests passed

Test Results

Submitted on 15 Feb 2017 by Will Roe | Solution ID: 3583120

```
function [u,status,iAl] = myMPController(H,G,F,bb,J,L,x,xTarget,m,iA)
% H       - quadratic term in the cost function.
% G       - matrix in the linear term in the cost function.
% F       - LHS of the inequalities term.
% bb      - RHS of the inequalities term.
% J       - RHS of inequalities term.
% L       - RHS of inequalities term. Use this to shift constraints around target point
% x       - current state
% xTarget - target equilibrium point.
% m       - Number of inputs.
% iA      - active inequalities, see doc mpcqpsolver
%
% u is the first input vector u_0 in the sequence [u_0; u_1; ...; u_(N-1)];
% In other words, u is the value of the receding horizon control law
% evaluated with the current state x0 and target xTarget

% Please read the documentation on mpcqpsolver to understand how it is
% suppose to be used. Use iA and iAl to pass the active inequality vector

opt = mpcqpsolverOptions;
opt.IntegrityChecks = false;%% for code generation
opt.FeasibilityTol = 1e-3;
opt.DataType = 'double';
%% your code starts here
Linv = H;

f = transpose(x-xTarget)*transpose(G);
f = transpose(f);

A = -F;
b = -bb + -J*x + -L*xTarget;
[U,status,iAl]=mpcqpsolver(Linv,f,A,b,[],zeros(0,1),iA,opt);

%% your remaining code here
u = U(1:m,:);

end

% Remember to copy and paste your code above to the Simulink model.
```

Next

- Cody Coursework
- Course
- People
- Catalog
- Eric Kerrigan
  - Log Out
- Help
- Course Details expand all
- Mathematical Model of a Gantry Crane
  - State space representation
  - Problem Explore
- Receding Horizon Control
  - Prediction Matrices
  - Cost Function Matrices
  - Linear RHC Law Without Constraints
  - Problem Explore
- RHC with Hard Constraints
  - Stage Constraints
  - Trajectory Constraints
  - QP Constraint Matrices
  - Receding Horizon Controller
  - Problem Explore
- RHC with Soft Constraints
  - Augmenting QP Matrices and Vectors
  - Problem Explore
- Assignment

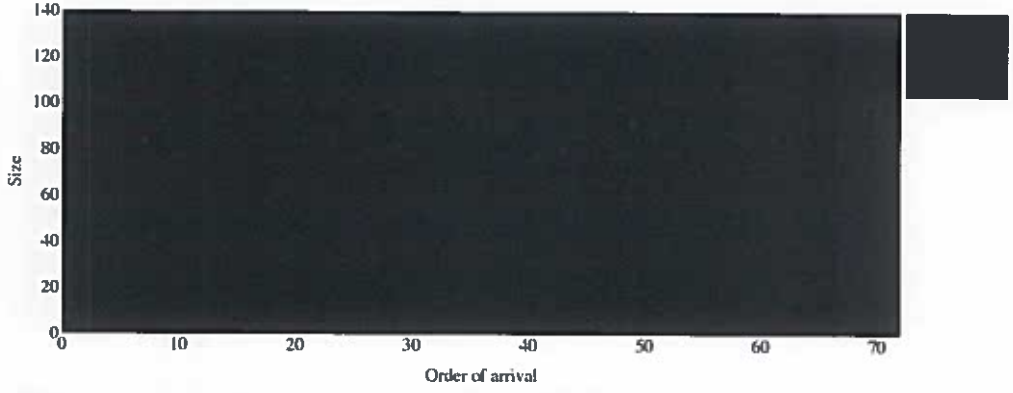
Receding Horizon Controller

Student Solutions

Search by student name, solution ID, or function Search by student name, solution ID, or function

- List
- Map

View: All Solutions Test Solutions Submitted Solutions



Previous

2nd Submission: All tests passed

Test Results

Submitted on 12 Feb 2017 by Fabian Duffhauß | Solution ID: 3528546

```
function [u,status,iAl] = myMPController(H,G,F,bb,J,L,x,xTarget,m,iA)
% H       - quadratic term in the cost function (Linv if using mpcqpsolver).
% G       - matrix in the linear term in the cost function.
% F       - LHS of the inequalities term.
% bb      - RHS of the inequalities term.
% J       - RHS of inequalities term.
% L       - RHS of inequalities term. Use this to shift constraints around target point
% x       - current state
% xTarget - target equilibrium point.
% m       - Number of inputs.
% iA      - active inequalities, see doc mpcqpsolver
%
% u is the first input vector u_0 in the sequence {u_0; u_1; ...; u_(N-1)};
% In other words, u is the value of the receding horizon control law
% evaluated with the current state x0 and target xTarget

% Please read the documentation on mpcqpsolver to understand how it is
% suppose to be used. Use iA and iAl to pass the active inequality vector

opt = mpcqpsolverOptions;
opt.IntegrityChecks = false;%% for code generation
opt.FeasibilityTol = 1e-3;
opt.DataType = 'double';

f = ((x - xTarget)' * G')';
b = -(bb + J * x + L * xTarget);
[U,status,iAl]=mpcqpsolver(H, f, -F, b, [], zeros(0, 1), iA, opt);
u = U(1:m);

end

% Remember to copy and paste your code above to the Simulink model.
```

Next

- Cody Coursework
- Course
- People
- Catalog
- Eric Kerrigan
  - Log Out
- Help
- Course Details expand all
- Mathematical Model of a Gantry Crane
  - State space representation
  - Problem Explore
- Receding Horizon Control
  - Prediction Matrices
  - Cost Function Matrices
  - Linear RHC Law Without Constraints
  - Problem Explore
- RHC with Hard Constraints
  - Stage Constraints
  - Trajectory Constraints
  - QP Constraint Matrices
  - Receding Horizon Controller
  - Problem Explore
- RHC with Soft Constraints
  - Augmenting QP Matrices and Vectors
  - Problem Explore
- Assignment

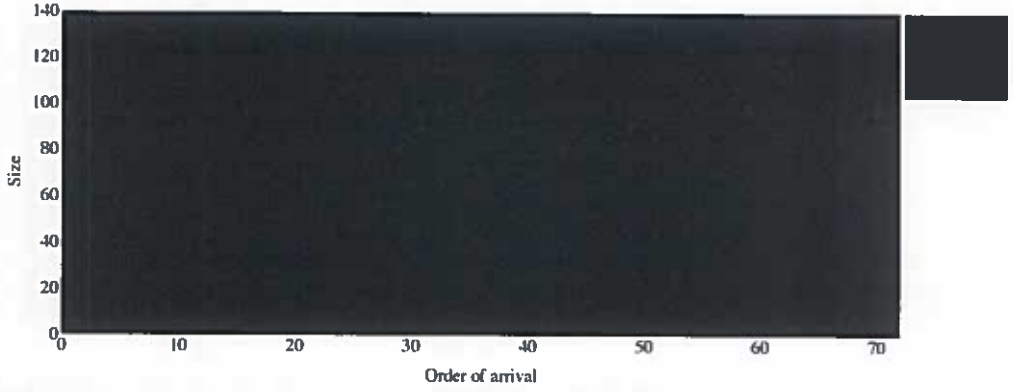
Receding Horizon Controller

Student Solutions

Search by student name, solution ID, or function Search by student name, solution ID, or function

- List
- Map

View: All Solutions Test Solutions Submitted Solutions



Previous

1st Submission: All tests passed

Test Results

Submitted on 10 Feb 2017 by Alina Walch | Solution ID: 3513442

```
function [u,status,iAl] = myMPController(H,G,F,bb,J,L,x,xTarget,m,iA)
% H       - quadratic term in the cost function.
% G       - matrix in the linear term in the cost function.
% F       - LHS of the inequalities term.m
% bb      - RHS of the inequalities term.
% J       - RHS of inequalities term.
% L       - RHS of inequalities term. Use this to shift constraints around target point
% x       - current state
% xTarget - target equilibrium point.
% m       - Number of inputs.
% iA      - active inequalities, see doc mpcqpsolver
%
% u is the first input vector u_0 in the sequence {u_0; u_1; ...; u_{N-1}};
% In other words, u is the value of the receding horizon control law
% evaluated with the current state x0 and target xTarget

% Please read the documentation on mpcqpsolver to understand how it is
% suppose to be used. Use iA and iAl to pass the active inequality vector

opt = mpcqpsolverOptions;
opt.IntegrityChecks = false;%% for code generation
opt.FeasibilityTol = 1e-3;
opt.DataType = 'double';
%% your code starts here
[U,status,iAl]=mpcqpsolver(H,G*(x-xTarget),-F,-bb-J*x-L*xTarget,[],zeros(0,1),iA,opt);
% [x,status,iA] = mpcqpsolver(Linv,f,A,b,Aeq,beq,iA0,options)
%% your remaining code here
u = U(1:m);
end

% Remember to copy and paste your code above to the Simulink model.
```

Next